Recursividad en JavaScript: Guía práctica con ejemplos resueltos

1. ¿Qué es la recursividad?

La recursividad es una técnica de programación en la que una función se llama a sí misma para resolver un problema descomponiéndolo en subproblemas más pequeños del mismo tipo. Dos elementos son imprescindibles: (a) caso base, que detiene la recursión, y (b) caso recursivo, que reduce el problema hacia el caso base.

2. Pila de llamadas (call stack) y riesgo de desbordamiento

Cada llamada a función se apila en la "call stack". Si la recursión no llega a un caso base o éste tarda demasiado, se puede producir un error de desbordamiento de pila (RangeError: Maximum call stack size exceeded). Por eso es clave asegurar el progreso hacia el caso base.

3. Patrón general de una función recursiva

```
function recursivo(problema) {
  if (esCasoBase(problema)) {
    return solucionTrivial(problema);
  }
  // Dividir o reducir el problema
  const subProblema = transforma(problema);
  const subSolucion = recursivo(subProblema);
  // Combinar resultados
  return combina(subSolucion);
}
```

---

4. Ejemplos resueltos

4.1. Factorial (n!)

Definición: n! = n × (n-1)! con 0! = 1. Es un ejemplo clásico para ilustrar el caso base y el caso recursivo.

```
function factorial(n) {
  if (n < 0) throw new Error('n debe ser >= 0');
  if (n === 0) return 1; // caso base
  return n * factorial(n - 1); // caso recursivo
}

console.log(factorial(5)); // 120
```

IES MARIA ENRIQUEZ

4.2. Fibonacci

La versión recursiva de Fibonacci

```
function fib(n, memo = {}) {
  if (n < 0) throw new Error('n debe ser >= 0');
  if (n <= 1) return n; // 0, 1
  if (memo[n] !== undefined) return memo[n];
  memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
  return memo[n];
}

console.log(fib(10)); // 55
```

4.3. Suma de un arreglo

Sumar elementos de un arreglo de forma recursiva, reduciendo el tamaño del arreglo en cada paso.

```
function suma(arr) {
  if (arr.length === 0) return 0; // caso base
  const [primero, ...resto] = arr;
  return primero + suma(resto);
}

console.log(suma([2, 3, 5])); // 10
```

---

5. Errores comunes

• No definir un caso base correcto.
• No progresar hacia el caso base (nunca decrecer el problema).
• Recalcular subproblemas.
• Usar recursión donde un bucle es más simple y eficiente.

6. Ejercicios adicionales (con solución)

6.1. Invertir una cadena

```
function invertirCadena(s) {
  if (s.length <= 1) return s;
  return invertirCadena(s.slice(1)) + s[0];
}

console.log(invertirCadena('recursion')); // 'noisrucer'
```

IES MARIA ENRIQUEZ

6.2. Profundidad máxima de un árbol binario

```
function profundidad(nodo) {
  if (!nodo) return 0;
  return 1 + Math.max(profundidad(nodo.izq),
profundidad(nodo.der));
}
```

6.3. Comprobar si una cadena es palíndromo

```
function esPalindromo(s) {
  const limpio = s.toLowerCase().replace(/[^a-z0-
9áéíóúüñ]/g, '');
  function aux(i, j) {
    if (i >= j) return true;
    if (limpio[i] !== limpio[j]) return false;
    return aux(i + 1, j - 1);
  }
  return aux(0, limpio.length - 1);
}

console.log(esPalindromo('Anita lava la tina')); // true
```

IES MARIA ENRIQUEZ